# CURSO ADMINISTRACION POSTGRESQL 18

## Módulo 1: Instalación y Arquitectura de PostgreSQL 17

**Meta:** Aprender a instalar PostgreSQL 17 en diferentes sistemas operativos y comprender su arquitectura interna.

### 1.1. Novedades Clave de PostgreSQL 17 para Administradores

Antes de instalar, es crucial entender qué hace especial a esta versión:

- **Rendimiento de `VACUUM`:** Nuevo sistema de gestión de memoria que reduce el consumo y mejora la velocidad. Ya no está limitado silenciosamente a 1 GB de memoria .
- **Gestión de Conexiones:** Nuevo modo de negociación SSL (`sslnegotiation=direct`) que reduce la latencia en las conexiones .
- **Particionamiento y Planificación:** Mejoras significativas en la optimización de consultas para tablas particionadas y el uso de índices .

### 1.2. Instalación desde Fuentes y Repositorios

- Compilación en Linux (Ubuntu/Debian, RHEL/Rocky).
- Configuración de variables de entorno (`PATH`, `PGDATA`, `PGPORT`).

### 1.3. Estructura de Archivos y Procesos

- **`PGDATA`:** El corazón de la instalación (bases de datos, configuraciones, WAL).
- **Procesos:** `Postmaster`, `Writer`, `WAL Writer`, `Checkpointer`, `Stats Collector` (nuevas vistas en 17, como `pg_stat_checkpointer`) .

---

## Módulo 2: Gestión de Usuarios y Roles

**Meta:** Implementar un control de acceso granular mediante Roles.

### 2.1. Roles vs. Usuarios

En PostgreSQL, no existe la diferencia tradicional de otros motores; un "usuario" es simplemente un rol con privilegio `LOGIN`.

### 2.2. Creación y Gestión de Roles

sql

```
-- Crear un rol de administrador (sin login)
CREATE ROLE admin_grp;

-- Crear un usuario de aplicación
CREATE USER app_user WITH PASSWORD 'secure_password' VALID UNTIL '2025-12-31';
```

### 2.3. Herencia de Privilegios (`INHERIT`)

Los roles pueden heredar privilegios de otros roles, simplificando la gestión masiva de permisos.

sql

```
GRANT admin_grp TO carlos;
```

### 2.4. Novedad en PostgreSQL 17: Rol `pg_maintain`

Anteriormente, ejecutar `VACUUM`, `ANALYZE` o `REINDEX` requería ser superusuario o el dueño de la tabla. Ahora puedes delegar esta tarea crítica de mantenimiento usando el rol predefinido `pg_maintain` o el permiso específico `MAINTAIN` .

sql

```
-- Ejemplo práctico (PostgreSQL 17+)
GRANT pg_maintain TO operador_dba;
-- O a nivel de tabla específica:
GRANT MAINTAIN ON TABLE facturas_2024 TO operador_dba;
```

---

## Módulo 3: Seguridad y Autorizaciones

**Meta:** Proteger los datos mediante autenticación robusta y control de acceso a nivel de fila/columna.

### 3.1. El Archivo `pg_hba.conf` (Autenticación)

- Métodos: `trust`, `password`, `scram-sha-256` (recomendado), `cert`, `gss`.
- **PostgreSQL 17:** Ahora genera logs detallados incluso para conexiones exitosas con método `trust`, mejorando la auditoría .

### 3.2. Privilegios Modernos (`GRANT`)

- `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- `USAGE` (para esquemas y secuencias).
- `MAINTAIN` (Nuevo en 17): Permite `VACUUM`, `ANALYZE`, `REINDEX`, `REFRESH MATERIALIZED VIEW` sin ser dueño .

### 3.3. Seguridad a Nivel de Fila (RLS)

Políticas que determinan qué filas puede ver o modificar un usuario.

### 3.4. `search_path` Seguro (Cambio crítico en 17)

En versiones anteriores, funciones usadas en índices podían ser "hackeadas" modificando el `search_path`. En **PostgreSQL 17**, las operaciones de mantenimiento (`VACUUM`, `REINDEX`) usan un `search_path` seguro, ignorando el del usuario que las ejecuta .

---

## Módulo 4: Mantenimiento y Optimización (`VACUUM` y `ANALYZE`)

**Meta:** Mantener la salud de la base de datos controlando la "hinchazón" (bloat) y la actualización de estadísticas.

### 4.1. El Problema de la Multiversión (MVCC)

Cómo se generan las tuplas muertas y por qué ralentizan las consultas.

### 4.2. `VACUUM` y `AUTOVACUUM`

- **`VACUUM`:** Marca espacio para reutilizar.
- **`VACUUM FULL`:** Re-escribe la tabla (bloquea).
- **`AUTOVACUUM`:** Daemon automático.

### 4.3. Mejoras en VACUUM (PostgreSQL 17)

- **Memoria Dinámica:** Ahora respeta rigurosamente `maintenance_work_mem` incluso para tablas enormes.
- **Monitoreo Mejorado:** La vista `pg_stat_progress_vacuum` ahora muestra `indexes_total` e `indexes_processed` .

### 4.4. `ANALYZE`

Recolecta estadísticas para el planificador de consultas.

---

## Módulo 5: Copias de Seguridad (Backup) y Recuperación (PITR)

**Meta:** Implementar estrategias de respaldo que minimicen la pérdida de datos.

### 5.1. Backup Lógico (`pg_dump` / `pg_dumpall`)

Ideal para migraciones o backups pequeños.

### 5.2. Backup Físico (Base de datos completa)

Uso de `pg_basebackup`.

### 5.3. Novedad Estrella: Backup Incremental (PostgreSQL 17)

PostgreSQL 17 introduce la capacidad de realizar **backups incrementales a nivel de archivo** de manera nativa, algo que antes requería herramientas externas como `pgBackRest` .

- **Procedimiento:**

1. Base backup (Full).
2. Backup incremental (solo cambios).

- **Utilidad:** `pg_combinebackup` para reconstruir el backup completo.

### 5.4. Recuperación Punto-a-Tiempo (PITR)

Uso de WAL (Write Ahead Logs) y `recovery.conf` (integrado ahora en `postgresql.conf`).

---

## Módulo 6: Réplica Lógica y Física

**Meta:** Configurar alta disponibilidad y escalabilidad de lecturas.

### 6.1. Streaming Replication (Física)

Bloque por bloque. Ideal para failover y lecturas consistentes.

### 6.2. Logical Replication

Publicación/Suscripción de datos a nivel de fila. Permite actualizaciones cruzadas entre versiones o bases de datos específicas.

### 6.3. Mejoras en Replicación (PostgreSQL 17)

- **`pg_createsubscriber`:** Nueva utilidad para convertir un servidor físico (standby) en un nodo de réplica lógica, sin necesidad de configuraciones complejas manuales .
- **`pg_upgrade` con Slots:** Actualizar una versión mayor de PostgreSQL ya no rompe los slots de replicación lógica .

---

## Módulo 7: Monitoreo y Auditoría

**Meta:** Observar el rendimiento y auditar quién hace qué en la BD.

### 7.1. Estadísticas en Tiempo Real

- **`pg_stat_activity`:** Conexiones activas, consultas bloqueantes.
- **`pg_stat_user_tables`:** Escaneos, lecturas, escrituras.
- **`pg_stat_bgwriter` y `pg_stat_checkpointer` (Nueva en 17):** Mide el rendimiento del escritor en segundo plano .

### 7.2. Vista `pg_wait_events` (Nueva en 17)

Documenta y permite consultar los eventos de espera que causan lentitud, facilitando el diagnóstico de cuellos de botella .

### 7.3. Logging Avanzado

- `log_duration`, `log_min_duration_statement`.
- Extensiones: `pgAudit` (para auditoría regulatoria tipo GDPR/SOX).

---

## Módulo 8: Parámetros Avanzados y Ajustes de 17

**Meta:** Ajustar el motor para casos de uso extremos.

### 8.1. Nuevos Parámetros de Configuración en 17

- **`transaction_timeout`:** Cancela automáticamente transacciones que duren más de X milisegundos. Ideal para evitar transacciones huérfanas .
- **`allow_alter_system`:** Permite deshabilitar completamente el comando `ALTER SYSTEM` si buscas una configuración estrictamente inmutable .
- **`huge_pages_status`:** Muestra si realmente se están utilizando "Huge Pages" a nivel de SO .

**8.2. Optimización de Memoria (SLRUs)**
PostgreSQL 17 permite configurar los buffers de caché para estructuras internas (`commit_timestamp_buffers`, `notify_buffers`, etc.), dando control fino sobre el uso de RAM en sistemas especializados .

**8.3. Buenas Prácticas Finales**
Checklist de seguridad y rendimiento para poner en producción.

---

## Proyecto Final Práctico

**Escenario:** Eres el DBA de una empresa financiera. Tienes que:

1. Instalar Postg**Módulo 1: Instalación y Arquitectura de PostgreSQL 17**reSQL 17 en un servidor Ubuntu 24.04.
2. Crear una base de datos llamada `finanzas` con un esquema `contabilidad`.
3. Crear los roles:

- `auditor` (solo lectura).
- `contable` (CRUD en tablas de `contabilidad`).
- `mantenimiento` (solo permisos de `VACUUM` y `ANALYZE` usando `pg_maintain`).

4. Configurar `pg_hba.conf` para permitir solo conexiones SSL con contraseña `scram-sha-256`.
5. Ejecutar un backup completo y luego un **backup incremental** (usando la nueva feature de 17).
6. Configurar una réplica lógica de una tabla crítica a otra base de datos llamada `finanzas_replica`.

---

## Recursos de Estudio Adicionales

- **Documentación Oficial:** Capítulos 20 (Database Roles), 21 (Privileges) y 19 (Server Configuration) de la documentación de PostgreSQL 17.
- **Comandos Clave:** `psql` con meta-comandos (`\l`, `\du`, `\dp`, `\x`).
- **Herramientas UI:** pgAdmin 4 (verifica que soporte 17) o DBeaver.

Desarrollame el punto 1.3 y la configuracion de variables de entorno. actualizado a postgreSQL 18.

Leer 10 páginas web

# Módulo 1:Arquitectura de PostgreSQL 18

A continuación, se desarrolla el punto **1.3. Estructura de Archivos y Procesos** junto con la **Configuración de Variables de Entorno**, completamente actualizado para **PostgreSQL 18**.

Basado en la documentación oficial de PostgreSQL 18 y fuentes complementarias, este material cubre en detalle la anatomía del servidor y la configuración del entorno del sistema operativo.

---

## Módulo 1.3: Estructura de Archivos y Procesos en PostgreSQL 18

Entender la arquitectura interna y la estructura de archivos de PostgreSQL es fundamental para cualquier tarea de administración, desde la resolución de problemas de espacio hasta la implementación de copias de seguridad a nivel de sistema de archivos.

### 1.3.1. El Directorio de Datos (`PGDATA`)

Todo comienza con el directorio de datos, conocido convencionalmente como `PGDATA`. Este directorio es el contenedor raíz de toda la información de un "cluster de bases de datos" (un término de PostgreSQL para referirse a una instancia del servidor que gestiona múltiples bases de datos) .

Cuando inicializas un cluster con `initdb -D /ruta/a/tu/directorio`, se crea este directorio con una estructura específica y permisos restrictivos (normalmente 0700) para garantizar la seguridad .

**Ubicación común:** `/var/lib/postgresql/18/main` (en sistemas Debian/Ubuntu) o `/usr/local/pgsql/data` (compilaciones desde fuente) .

### 1.3.2. Contenido del Directorio `PGDATA`

Al listar el contenido de `PGDATA`, te encontrarás con una serie de archivos y subdirectorios. Cada uno tiene un propósito crítico.

| Componente | Tipo | Descripción en PostgreSQL 18 |
|---|---|---|
| **`PG_VERSION`** | Archivo | Un archivo de texto plano que contiene el número de versión mayor de PostgreSQL (ej. `18`). El servidor lo verifica al arrancar . |
| **`base/`** | Subdirectorio | Es el lugar donde residen **todos los datos de tus bases de datos** (excepto los que están en tablespaces personalizados). Cada base de datos tiene una subcarpeta aquí, nombrada con su OID (Object Identifier) numérico . |
| **`global/`** | Subdirectorio | Contiene las tablas que son compartidas por todo el cluster, como `pg_database` (la lista de todas las BBDD) y `pg_authid` (la información de roles/usuarios) . |
| **`pg_wal/`** | Subdirectorio | **El corazón de la durabilidad.** Aquí se almacenan los archivos de Write-Ahead Log (WAL), también conocidos como registros de preescritura o "WAL logs". Son esenciales para la recuperación ante caídas y la replicación . |
| **`pg_tblspc/`** | Subdirectorio | Contiene enlaces simbólicos que apuntan a los directorios de los **tablespaces** definidos por el usuario. Esto permite ubicar datos en discos o sistemas de archivos diferentes . |

| Componente | Tipo | Descripción en PostgreSQL 18 |
|---|---|---|
| **`pg_logical/`** | Subdirectorio | Almacena metadatos y estados para la **replicación lógica** (publicaciones, suscripciones, slots de replicación) . |
| **`postgresql.conf`** | Archivo | El archivo de configuración principal del servidor. Controla casi todos los parámetros de comportamiento, memoria y recursos . |
| **`pg_hba.conf`** | Archivo | El archivo de control de acceso. Define qué clientes pueden conectarse a qué bases de datos y qué método de autenticación deben usar (ej. `scram-sha-256`) . |
| **`postgresql.auto.conf`** | Archivo | Un archivo de configuración gestionado por el comando `ALTER SYSTEM`. Sus valores sobrescriben a los de `postgresql.conf` y es útil para cambios automatizados . |
| **`postmaster.pid`** | Archivo | Un archivo de bloqueo (lock file) que existe mientras el servidor está en ejecución. Contiene el PID del proceso principal, el puerto y el directorio de datos, evitando que se inicie otra instancia en el mismo directorio . |

### 1.3.3. Estructura Interna de Almacenamiento de Tablas e Índices

Profundizando en el subdirectorio `base/`, la forma en que PostgreSQL guarda una tabla o índice es muy específica:

- **Archivos por Relación:** Cada tabla e índice se almacena en uno o más archivos dentro del subdirectorio de su base de datos.
- **Nodo de Archivo (`filenode`):** El nombre de estos archivos se basa en el `relfilenode` de la relación (obtenible de `pg_class.relfilenode`), que puede cambiar tras operaciones como `TRUNCATE` o `REINDEX` .
- **Segmentación de 1 GB:** Cuando un archivo supera 1 GB, PostgreSQL lo divide automáticamente en segmentos: `relfilenode`, `relfilenode.1`, `relfilenode.2`, etc. Esto evita problemas con sistemas de archivos antiguos .
- **Archivos Auxiliares (`fork`):** Además del archivo de datos principal, existen:
- `_fsm` (Free Space Map): Mapa de espacio libre para encontrar rápidamente dónde escribir nuevos datos.
- `_vm` (Visibility Map): Mapa de visibilidad para acelerar `VACUUM` y el escaneo solo de índices (Index-Only Scans) .

### 1.3.4. La Arquitectura de Procesos (Multiproceso)

A diferencia de otros motores de bases de datos que son "multihilo", PostgreSQL 18 utiliza una arquitectura **"multiproceso" basada en el cliente/servidor"** .

1. **`postgres` (Proceso Supervisor):** Es el primer proceso que se inicia. También se le conoce como `postmaster`. Escucha en la red y puerto asignado (ej. 5432) esperando nuevas conexiones

de clientes. Cuando un cliente se conecta, el `postmaster` "forkea" (crea) un nuevo proceso hijo para atenderlo exclusivamente .

2. **Procesos Hijos (`postgres`):** Cada conexión de cliente (cada sesión en `psql`, cada `PgAdmin`, cada aplicación) recibe su propio proceso de servidor. Este proceso es el responsable de ejecutar todas las consultas SQL de esa conexión. Esto proporciona un gran aislamiento: si un proceso hijo falla (crash), el resto del sistema y otras conexiones no se ven afectadas.
3. **Procesos en Segundo Plano ("Background Workers"):** Además de los procesos de usuario, el `postmaster` lanza una serie de procesos internos vitales para el mantenimiento y rendimiento . Los más importantes son:

- **`checkpointer`:** Escribe todos los bloques de datos "sucios" (modificados) de la memoria compartida a los archivos del disco. Su objetivo es reducir la carga de trabajo del `writer`.
- **`background writer`:** Es un proceso más constante que va escribiendo bloques sucios al disco de forma periódica para evitar que el `checkpointer` tenga que hacer una gran cantidad de trabajo de golpe.
- **`walwriter`:** Se encarga de vaciar (flush) los registros del WAL (Write-Ahead Log) desde la memoria hacia los archivos dentro del directorio `pg_wal`. Es crítico para la durabilidad (persistencia de las transacciones confirmadas).
- **`autovacuum launcher`** / **`autovacuum worker`:** Lanza procesos de `VACUUM` automáticos para limpiar tuplas muertas (dead tuples) y actualizar estadísticas, previniendo la "hinchazón" (bloat) de las tablas.
- **`stats collector`:** Recopila estadísticas sobre la actividad del servidor (número de escaneos de tablas, índices usados, etc.), que se pueden consultar en las vistas `pg_stat_*`.

---

## Configuración de Variables de Entorno en PostgreSQL 18

Las variables de entorno son una forma estándar en sistemas Unix/Linux de pasar información a los programas sin usar argumentos de línea de comandos. Tanto el servidor como las herramientas cliente (`psql`, `pg_dump`) las utilizan.

### 1.3.5. La Variable Fundamental: `PGDATA`

Para el **servidor (`postgres`)**, la variable más importante es `PGDATA`. Si la defines, el servidor la usará como directorio de datos por defecto, evitando tener que especificar la opción `-D` cada vez .

**Cómo configurarla (para el usuario `postgres` en Bash):**
Edita el archivo `~/.bash_profile` o `~/.bashrc` del usuario `postgres`:

bash

```
export PGDATA=/var/lib/postgresql/18/main
```

### 1.3.6. Variables para Herramientas de Administración

- **`PATH`:** Para poder ejecutar comandos como `psql`, `pg_ctl` o `pg_dump` sin escribir la ruta completa, debes añadir el directorio `bin` de PostgreSQL a tu `PATH` .

  bash

  ```
  export PATH=/usr/lib/postgresql/18/bin:$PATH
  ```

- **`LD_LIBRARY_PATH` (Linux/Unix):** Si compilaste PostgreSQL desde el código fuente y lo instalaste en una ubicación no estándar (ej. `/opt/pgsql18`), es posible que los programas no encuentren las librerías compartidas (`libpq.so`). En ese caso, usa esta variable :

  bash

  ```
  export LD_LIBRARY_PATH=/opt/pgsql18/lib:$LD_LIBRARY_PATH
  ```

  *Nota:* En instalaciones mediante paquetes (`.deb`, `.rpm`), esto no suele ser necesario.

### 1.3.7. Variables para Clientes (`libpq`)

Todas las aplicaciones cliente que usan la librería `libpq` (la interfaz C de PostgreSQL) respetan una serie de variables de entorno para establecer la conexión por defecto. Esto es muy útil para no tener que escribir `-h localhost -p 5432 -U usuario` cada vez .

| Variable | Equivalente en `psql` | Descripción |
|---|---|---|
| **`PGHOST`** | `-h` | Dirección del servidor (`localhost` o una IP). |
| **`PGPORT`** | `-p` | Puerto del servidor (por defecto `5432`). |
| **`PGDATABASE`** | `-d` | Nombre de la base de datos a la que conectarse. |
| **`PGUSER`** | `-U` | Nombre del rol/usuario de PostgreSQL. |
| **`PGPASSWORD`** | (No recomendado) | Contraseña. **Inseguro**, mejor usar un archivo `.pgpass` . |

**Ejemplo práctico para un usuario de aplicaciones:**
En el archivo `~/.bashrc` de tu usuario del sistema operativo (ej. `aplicacion`):

bash

```
export PGHOST=localhost
export PGPORT=5432
export PGDATABASE=mi_app_db
export PGUSER=admin_app
```

Ahora, al ejecutar `psql`, se conectará automáticamente a `mi_app_db` con el usuario `admin_app` sin necesidad de argumentos.

### 1.3.8. Resumen de Configuración Recomendada

Un script de entorno robusto para un DBA que trabaje con PostgreSQL 18 podría verse así:

bash

```
# Archivo: ~/.postgres18_env

# 1. Configuración del Servidor (si eres el usuario 'postgres')
export PGDATA=/var/lib/postgresql/18/main
```

```
# 2. Configuración de PATH y Librerías (para encontrar los binarios)
export PATH=/usr/lib/postgresql/18/bin:$PATH
export MANPATH=/usr/share/postgresql/18/man:$MANPATH

# 3. Configuración de Cliente por Defecto (para conectarte fácilmente)
export PGHOST=localhost
export PGPORT=5432
export PGDATABASE=postgres   # Conectar a la BBDD por defecto
export PGUSER=postgres       # Tu rol de superusuario

# Opcional: Para cliente en modo verboso o editor
export PSQL_EDITOR=vim
export PAGER=less
```

Para activarlo, ejecuta `source ~/.postgres18_env`. Puedes usar la herramienta `pg_config` para verificar rutas de instalación si tienes dudas .

y ahora desarrollame el Módulo 2: Gestión de Usuarios y Roles actualizado a postgresql 18

Leer 10 páginas web

A continuación se desarrolla el **Módulo 2: Gestión de Usuarios y Roles** completamente actualizado para **PostgreSQL 18**.

Basado en la documentación oficial de PostgreSQL 18, este módulo cubre en detalle el modelo unificado de roles, la creación y gestión de usuarios, y las novedades más relevantes en materia de seguridad y administración.

---

# Módulo 2: Gestión de Usuarios y Roles en PostgreSQL 18

La gestión de usuarios y permisos en PostgreSQL 18 se basa en el concepto de **Roles**. Un rol puede ser un usuario individual (con capacidad de conexión) o un grupo (sin capacidad de conexión, usado para agrupar permisos). Esta unificación, que sigue el estándar SQL, simplifica enormemente la administración de privilegios .

---

### 2.1. Concepto de Rol: Unificación de Usuario y Grupo

En PostgreSQL, no existe la diferencia tradicional entre "usuario" y "grupo" que encuentras en otros motores. En su lugar, existe la entidad **Rol** .

- **Rol con `LOGIN`:** Es equivalente a un "usuario". Puede iniciar una sesión en la base de datos.
- **Rol sin `LOGIN`:** Es equivalente a un "grupo". Sirve como contenedor de permisos que luego se pueden asignar a otros roles.

**Dato clave para PostgreSQL 18:** La semántica de `INHERIT` ha sido refinada. Antes de PostgreSQL 16, `INHERIT` era un atributo del rol que controlaba todas las comprobaciones de membresía. Ahora, el comportamiento de herencia se controla de manera más granular a nivel de la membresía misma .

---

### 2.2. Creación y Gestión de Roles con `CREATE ROLE`

La instrucción fundamental es `CREATE ROLE`. Para ejecutarla, necesitas el privilegio `CREATEROLE` o ser superusuario .

**Sintaxis básica:**

sql

```
CREATE ROLE nombre_rol [WITH] opción ... ;
```

**Atributos Principales (Opciones `WITH`)**

Aquí tienes la tabla actualizada con todos los atributos disponibles en PostgreSQL 18:

| Atributo | Descripción | Valor por Defecto |
|---|---|---|
| `LOGIN` / `NOLOGIN` | Permite (o no) que el rol inicie sesión. `CREATE USER` es un alias que implica `LOGIN` . | `NOLOGIN` |
| `SUPERUSER` / `NOSUPERUSER` | El rol puede eludir **todos** los controles de acceso. **Extremadamente peligroso**. | `NOSUPERUSER` |
| `CREATEDB` / `NOCREATEDB` | Permite (o no) crear nuevas bases de datos. | `NOCREATEDB` |
| `CREATEROLE` / `NOCREATEROLE` | Permite (o no) crear, modificar y eliminar otros roles. **No otorga superusuario**. | `NOCREATEROLE` |
| `INHERIT` / `NOINHERIT` | Controla si el rol **hereda automáticamente** los privilegios de los roles de los que es miembro. En PG18, esto es más matizado (ver sección 2.3) . | `INHERIT` |
| `BYPASSRLS` / `NOBYPASSRLS` | Permite (o no) que el rol ignore las políticas de Seguridad a Nivel de Fila (RLS). | `NOBYPASSRLS` |
| `REPLICATION` / `NOREPLICATION` | Permite (o no) que el rol inicie conexiones de replicación (física o lógica). | `NOREPLICATION` |
| `CONNECTION LIMIT` | Número máximo de conexiones concurrentes para este rol (-1 = sin límite). | -1 |
| `PASSWORD 'password'` | Establece la contraseña. Solo relevante para roles con `LOGIN`. | Ninguna |
| `VALID UNTIL 'timestamp'` | Fecha de expiración de la **contraseña** (no del rol). | Sin expiración |

**Advertencia sobre contraseñas:**

1. **Evita MD5:** El soporte para contraseñas cifradas con MD5 está **obsoleto** y se eliminará en una versión futura. Usa `scram-sha-256` configurando `password_encryption = scram-sha-256` en `postgresql.conf` .
2. **Seguridad:** Al crear un rol con `PASSWORD`, la contraseña viaja en texto plano al servidor. En entornos productivos, usa el comando `\password` dentro de `psql` o la utilidad `createuser` .

**Ejemplos Prácticos:**

sql

```
-- 1. Crear un usuario de aplicación estándar
CREATE USER app_usuario WITH PASSWORD 'UnaBuen@Contraseña';

-- 2. Crear un usuario que caduca a fin de año
CREATE ROLE "usuario_temp" WITH LOGIN PASSWORD 'temp2024' VALID UNTIL '2024-12-
31';

-- 3. Crear un rol de grupo (sin login) para administradores de BD
CREATE ROLE grupo_admin WITH CREATEDB CREATEROLE BYPASSRLS;

-- 4. Crear un usuario con límite de 5 conexiones simultáneas
CREATE ROLE "usuario_limitado" WITH LOGIN CONNECTION LIMIT 5;
```

---

### 2.3. Membresía y Herencia: El Modelo Refinado de PG18

La forma de heredar privilegios ha cambiado ligeramente para ser más segura y estándar. La palabra clave `INHERIT` ahora actúa como un **valor predeterminado** para las nuevas membresías, no como una propiedad absoluta del rol .

**Conceptos Clave:**

1. **`INHERIT` / `NOINHERIT`:** Es un atributo **del rol que se añade como miembro** (el miembro). Define el comportamiento por defecto cuando este rol se une a otro.
2. **Opciones de membresía (`GRANT`):**

- `WITH INHERIT TRUE/FALSE`: Controla si, en esta membresía específica, los privilegios se heredan.
- `WITH SET TRUE/FALSE`: Permite (o no) usar `SET ROLE` para convertirse en ese rol.
- `WITH ADMIN TRUE/FALSE`: Permite (o no) otorgar la membresía a otros.

**Escenario Práctico:**

sql

```
-- 1. Crear un rol de grupo 'lectores' con acceso SELECT en una tabla
CREATE ROLE lectores;
GRANT SELECT ON tabla_informes TO lectores;

-- 2. Crear un usuario 'juan' que NO hereda privilegios por defecto
CREATE ROLE juan WITH LOGIN NOINHERIT;

-- 3. Añadir a 'juan' como miembro de 'lectores', pero FORZAR la herencia para
este grupo
GRANT lectores TO juan WITH INHERIT TRUE;

-- Resultado: 'juan' podrá leer 'tabla_informes' gracias a la membresía con
herencia,
-- aunque su rol tenga NOINHERIT por defecto.
```

---

### 2.4. Roles Predefinidos en PostgreSQL 18

PostgreSQL 18 incluye una serie de roles predefinidos que facilitan la delegación de tareas administrativas sin otorgar superusuario .

**Novedad en PostgreSQL 18:** El privilegio `MAINTAIN` y el rol `pg_maintain` se han mejorado. Ahora, `pg_maintain` también permite ejecutar **funciones de manipulación de estadísticas**, lo que amplía su utilidad para tareas de monitoreo y optimización .

**Tabla de Roles Predefinidos Esenciales:**

| Rol Predefinido | Capacidad que Otorga (Sin ser Superusuario) | Ejemplo de Uso |
|---|---|---|
| **`pg_maintain`** | Ejecutar `VACUUM`, `ANALYZE`, `REINDEX`, `CLUSTER`, `REFRESH MATERIALIZED VIEW` y **funciones de manipulación de estadísticas** . | `GRANT pg_maintain TO operador_mantenimiento;` |
| **`pg_monitor`** | Leer todas las vistas `pg_stat_*` y `pg_settings`. Es miembro de `pg_read_all_settings`, `pg_read_all_stats` y `pg_stat_scan_tables` . | `GRANT pg_monitor TO monitor_dba;` |
| **`pg_signal_backend`** | Enviar señales (`pg_cancel_backend`, `pg_terminate_backend`) a procesos de otros usuarios (excepto superusuarios) . | `GRANT pg_signal_backend TO admin_soporte;` |
| **`pg_read_all_data`** | Leer **todos** los datos de todas las tablas, vistas y secuencias. Útil para roles de respaldo o analítica . | `GRANT pg_read_all_data TO rol_analista;` |
| **`pg_write_all_data`** | Escribir (INSERT, UPDATE, DELETE) en **todas** las tablas. Usar con extrema precaución . | `GRANT pg_write_all_data TO rol_integracion;` |
| **`pg_use_reserved_connections`** | Usar las conexiones reservadas configuradas por `reserved_connections`. Crítico para usuarios administradores . | `GRANT pg_use_reserved_connections TO admin_soporte;` |

### 2.5. Herramientas de Línea de Comandos: `createuser` y `dropuser`

PostgreSQL proporciona utilidades que son wrappers de los comandos SQL .

**`createuser` (Ejemplos):**

bash

```bash
# Crear usuario 'juan' con prompts interactivos
createuser --interactive juan

# Crear usuario 'app_web' con contraseña (se solicitará de forma segura)
createuser -P app_web

# Crear usuario 'dba_lectura' con solo permisos de monitoreo
createuser --no-superuser --no-createdb --no-createrole --inherit --login \
  --member=pg_monitor dba_lectura

# Crear usuario en un host y puerto específicos
createuser -h db.miempresa.com -p 5433 -U admin_global nuevo_usuario
```

---

### 2.6. Modificación y Eliminación de Roles

Una vez creados, los roles se modifican con `ALTER ROLE` y se eliminan con `DROP ROLE`.

sql

```sql
-- Cambiar contraseña de forma segura (dentro de psql)
\password juan

-- Renombrar un rol
ALTER ROLE juan RENAME TO juan_perez;

-- Revocar un privilegio específico (ej: ya no puede crear BBDD)
ALTER ROLE juan_perez NOCREATEDB;

-- Eliminar un rol (debe estar sin objetos a su nombre)
DROP ROLE juan_perez;

-- Forzar eliminación (reasigna objetos a otro rol)
DROP ROLE juan_perez ASSIGN TO admin_general;
```

---

### 2.7. Buenas Prácticas para la Gestión de Roles en PostgreSQL 18

1. **Principio de Mínimo Privilegio:** Usa `CREATEROLE` en lugar de `SUPERUSER` para administradores de bases de datos. Un rol con `CREATEROLE` puede gestionar usuarios y grupos, pero no puede leer cualquier tabla ni eludir RLS .
2. **Usa Roles de Grupo:** No asignes permisos directamente a usuarios. Crea roles de grupo (sin `LOGIN`), concédeles los permisos, y luego haz a los usuarios miembros de esos grupos.
3. **Aprovecha `pg_maintain`:** Es la forma moderna y segura de delegar tareas de mantenimiento (como `VACUUM`) a personal de operaciones sin darles más poder del necesario .
4. **Audita con `pg_monitor`:** Crea un rol dedicado al que le otorgues `pg_monitor` y úsalo exclusivamente para conectar herramientas de monitoreo (Prometheus, Zabbix, etc.) .
5. **Protege las Conexiones:** Usa `CONNECTION LIMIT` para evitar que una aplicación mal comportada agote todas las conexiones del servidor.

6. **Gestiona el Ciclo de Vida:** Define políticas de caducidad de contraseñas con `VALID UNTIL` para cuentas temporales o de servicios externos.

---

### Resumen del Módulo 2

Has aprendido que en PostgreSQL 18:

- **Roles** son el concepto unificado para usuarios y grupos.
- La **herencia de privilegios** es más granular, controlada por opciones de membresía (`WITH INHERIT`).
- **Roles predefinidos** como `pg_maintain` y `pg_monitor` son herramientas poderosas para delegar tareas administrativas de forma segura, con la novedad de que `pg_maintain` ahora incluye manipulación de estadísticas.
- Las utilidades de línea de comandos (`createuser`, `dropuser`) son complementos prácticos para la administración diaria.
- Las contraseñas deben gestionarse con `scram-sha-256` y cambiarse con `\password` en `psql`.

y ahora el modulo 3

Leer 10 páginas web

A continuación se desarrolla el **Módulo 3: Seguridad y Autorizaciones** completamente actualizado para **PostgreSQL 18**.

Basado en la documentación oficial y fuentes especializadas, este módulo cubre las novedades más importantes en autenticación y control de acceso, incluyendo OAuth 2.0, roles predefinidos mejorados, y políticas de seguridad a nivel de fila.

---

# Módulo 3: Seguridad y Autorizaciones en PostgreSQL 18

La seguridad en PostgreSQL 18 ha recibido mejoras significativas, destacando especialmente la introducción de **OAuth 2.0** como método de autenticación nativo, la consolidación del privilegio `MAINTAIN`, y nuevas opciones para el control de acceso fino.

---

### 3.1. El Archivo `pg_hba.conf` (Autenticación)

El corazón de la autenticación en PostgreSQL sigue siendo el archivo `pg_hba.conf` (Host-Based Authentication). Este archivo controla qué clientes pueden conectarse, a qué bases de datos, y qué método de autenticación deben usar.

**Métodos de Autenticación en PostgreSQL 18**

| Método | Descripción | Recomendación |
|---|---|---|
| `trust` | Permite conexión sin contraseña. **Peligroso**. | Solo para entornos de desarrollo locales. |
| `password` | Envía contraseña en texto plano. | **No usar** en producción. |
| `scram-sha-256` | Autenticación segura con contraseña cifrada. | **Recomendado para contraseñas**. |
| `cert` | Autenticación mediante certificados SSL/TLS. | Para conexiones de servicio seguro. |
| `gss` | Autenticación Kerberos/GSSAPI. | Entornos corporativos con Active Directory. |
| `oauth` | **NUEVO en PostgreSQL 18** - Autenticación mediante OAuth 2.0 . | Ideal para integración con proveedores de identidad. |

**Novedad Estrella: Autenticación OAuth 2.0 (PostgreSQL 18)**

PostgreSQL 18 introduce soporte nativo para OAuth 2.0, permitiendo que aplicaciones y usuarios utilicen tokens de identidad de proveedores como Google, Auth0, o servidores OAuth corporativos .

**Configuración en `pg_hba.conf`**:

bash

```
# TYPE  DATABASE  USER    ADDRESS        METHOD              PARAMETERS
host    mydb      all     0.0.0.0/0      oauth
issuer=https://oauth-provider.com scope="openid profile email"
host    appdb     app_role 0.0.0.0/0     oauth
issuer=https://auth.company.com scope="api:db" map=oauth_map
```

**Parámetros OAuth**:

| Parámetro | Descripción | Obligatorio |
|---|---|---|
| `issuer` | URL del servidor de autorización (proveedor OAuth) | Sí |
| `scope` | Lista de alcances OAuth separados por espacio | Sí |
| `validator` | Nombre del validador (si hay múltiples librerías) | No |
| `map` | Mapeo de nombres de proveedor a usuarios de BD | No |

**Conexión con `psql`**:

bash

```
# Conexión interactiva (abre navegador/web para autenticación)
psql "host=mydb.com dbname=appdb oauth_issuer=https://oauth-provider.com
oauth_client_id=my-client-id"

# Conexión con token preexistente
psql "host=mydb.com dbname=appdb oauth_issuer=https://oauth-provider.com
oauth_client_id=my-client-id oauth_token=opaque-token"
```

**Ventaja clave:** Elimina la necesidad de gestionar contraseñas estáticas de base de datos, centralizando la identidad en un proveedor externo .

#### Configuración de `scram-sha-256` (Recomendada)

Para usar el método más seguro con contraseñas, configura en `postgresql.conf`:

ini

```
password_encryption = scram-sha-256
```

Ejemplo de entrada en `pg_hba.conf`:

bash

```
# TYPE  DATABASE    USER        ADDRESS         METHOD
host    all         all         192.168.1.0/24  scram-sha-256
host    all         all         ::1/128         trust    # solo localhost IPv6
```

---

### 3.2. Roles Predefinidos y Privilegios en PostgreSQL 18

PostgreSQL 18 mantiene y mejora el sistema de roles predefinidos introducido en versiones anteriores. Estos roles permiten delegar tareas administrativas sin otorgar superusuario.

#### Tabla de Roles Predefinidos en PostgreSQL 18

| Rol Predefinido | Capacidad | Uso Típico |
|---|---|---|
| **`pg_maintain`** | Ejecuta `VACUUM`, `ANALYZE`, `REINDEX`, `CLUSTER`, `REFRESH MATERIALIZED VIEW` en todas las relaciones . | Equipo de operaciones/DBA |
| **`pg_checkpoint`** | Permite ejecutar el comando `CHECKPOINT` . | Monitoreo y mantenimiento |
| **`pg_monitor`** | Lee todas las vistas `pg_stat_*` y `pg_settings`. Miembro de `pg_read_all_settings`, `pg_read_all_stats`, `pg_stat_scan_tables` . | Herramientas de monitoreo |
| **`pg_signal_backend`** | Cancela consultas o termina sesiones de otros usuarios . | Soporte técnico |
| **`pg_signal_autovacuum_worker`** | Señala a workers de autovacuum para cancelar su trabajo actual . | Administración avanzada |
| **`pg_read_all_data`** | Lee **todos** los datos de todas las tablas, vistas y secuencias . | Respaldos, analítica |
| **`pg_write_all_data`** | Escribe en **todas** las tablas (INSERT, UPDATE, DELETE) . | Integración de datos |
| **`pg_read_server_files`** | Lee archivos del servidor mediante `COPY` y funciones relacionadas . | Carga de datos externos |
| **`pg_write_server_files`** | Escribe archivos en el servidor . | Exportación de datos |
| **`pg_execute_server_prog`** | Ejecuta programas en el servidor . | **Usar con extrema precaución** |

| Rol Predefinido | Capacidad | Uso Típico |
|---|---|---|
| `ram` | | |
| `pg_use_reserved_connections` | Usa conexiones reservadas (`reserved_connections`) . | Administradores de emergencia |

**Advertencia**: Los roles `pg_read_server_files`, `pg_write_server_files` y `pg_execute_server_program` otorgan acceso directo al sistema de archivos y pueden usarse para escalar privilegios. **Conceder solo a roles completamente confiables** .

**Ejemplo de concesión**:

sql

```
-- Crear un rol de monitor
CREATE ROLE monitor WITH LOGIN PASSWORD 'secure_pass';
GRANT pg_monitor TO monitor;

-- Crear un rol de mantenimiento
CREATE ROLE mantenimiento WITH LOGIN PASSWORD 'maintain_pass';
GRANT pg_maintain TO mantenimiento;

-- Otorgar solo a un usuario específico
GRANT pg_signal_backend TO admin_soporte;
```

#### El Privilegio `MAINTAIN` (Reforzado en PG18)

El privilegio `MAINTAIN` se reintrodujo en versiones recientes de PostgreSQL tras solucionar problemas de seguridad relacionados con `search_path` . Permite ejecutar operaciones de mantenimiento sin ser propietario de la tabla.

sql

```
-- Otorgar privilegio MAINTAIN en una tabla específica
GRANT MAINTAIN ON TABLE facturas TO operador_mantenimiento;

-- Revocar
REVOKE MAINTAIN ON TABLE facturas FROM operador_mantenimiento;
```

#### Nuevo: Privilegios `SET` y `ALTER SYSTEM`

PostgreSQL 18 permite controlar quién puede modificar parámetros de configuración:

sql

```
-- Permite a un rol usar SET para cambiar un parámetro
GRANT SET ON PARAMETER work_mem TO rol_app;

-- Permite a un rol usar ALTER SYSTEM (¡peligroso!)
GRANT ALTER SYSTEM ON PARAMETER max_connections TO dba_parametros;
```

---

### 3.3. Seguridad a Nivel de Fila (RLS)

RLS (Row Level Security) permite controlar el acceso a nivel de fila mediante **políticas (policies)**. PostgreSQL 18 mantiene la sintaxis madura de versiones anteriores .

#### Activación de RLS

sql

```
-- Habilitar RLS en una tabla
ALTER TABLE usuarios ENABLE ROW LEVEL SECURITY;

-- Forzar RLS (ni siquiera el propietario puede saltarlo)
ALTER TABLE usuarios FORCE ROW LEVEL SECURITY;
```

#### Creación de Políticas

La sintaxis `CREATE POLICY` permite definir condiciones para SELECT, INSERT, UPDATE y DELETE :

sql

```
CREATE POLICY policy_name ON table_name
    [ AS { PERMISSIVE | RESTRICTIVE } ]
    [ FOR { ALL | SELECT | INSERT | UPDATE | DELETE } ]
    [ TO role_name [, ...] ]
    [ USING (using_expression) ]
    [ WITH CHECK (check_expression) ];
```

**Conceptos Clave**:

| Cláusula | Cuándo se Aplica | Propósito |
|---|---|---|
| `USING` | SELECT, UPDATE (lectura), DELETE | Filtra **qué filas son visibles** |
| `WITH CHECK` | INSERT, UPDATE (escritura) | Valida **si la nueva fila es permitida** |

#### Tipos de Políticas: PERMISSIVE vs RESTRICTIVE

| Tipo | Comportamiento | Uso |
|---|---|---|
| **PERMISSIVE** (por defecto) | Combina condiciones con **OR**. Amplía el acceso. | Políticas "habilitadoras" |
| **RESTRICTIVE** | Combina condiciones con **AND**. Reduce el acceso. | Políticas "limitadoras" |

**Importante**: Si solo hay políticas RESTRICTIVE sin ninguna PERMISSIVE, no se podrá acceder a **ninguna** fila .

#### Ejemplos Prácticos de RLS

### Escenario 1: Multi-tenencia por columna `tenant_id`

sql

```
-- Tabla de pedidos con tenant_id
CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER NOT NULL,
    cliente TEXT,
    monto NUMERIC
);

-- Activar RLS
ALTER TABLE pedidos ENABLE ROW LEVEL SECURITY;

-- Política: solo ver pedidos de su tenant
```

```
CREATE POLICY pedidos_tenant_policy ON pedidos
    USING (tenant_id = current_setting('app.current_tenant')::INTEGER);

-- Política para INSERT: solo insertar con su tenant
CREATE POLICY pedidos_insert_policy ON pedidos
    FOR INSERT
    WITH CHECK (tenant_id = current_setting('app.current_tenant')::INTEGER);
```

**Escenario 2: Usuarios solo ven sus propios datos**

sql

```
CREATE TABLE documentos (
    id SERIAL,
    owner TEXT DEFAULT CURRENT_USER,
    contenido TEXT,
    publicado BOOLEAN DEFAULT FALSE
);

ALTER TABLE documentos ENABLE ROW LEVEL SECURITY;

-- Política permisiva: documentos propios o públicos
CREATE POLICY doc_permisiva ON documentos
    AS PERMISSIVE
    USING (owner = CURRENT_USER OR publicado = TRUE);

-- Política restrictiva: NO ver documentos archivados
CREATE POLICY doc_restrictiva ON documentos
    AS RESTRICTIVE
    USING (contenido NOT LIKE '%[ARCHIVADO]%');
```

**Escenario 3: RLS con roles predefinidos**

sql

```
-- Los auditores pueden ver todo, pero no modificar
GRANT pg_read_all_data TO rol_auditor;

-- Los usuarios normales tienen RLS
ALTER TABLE datos_sensibles ENABLE ROW LEVEL SECURITY;

CREATE POLICY datos_por_region ON datos_sensibles
    TO PUBLIC
    USING (region = current_setting('app.user_region'));
```

**Consideraciones de Rendimiento para RLS**

1. **Indexar columnas de política**: Las columnas usadas en `USING` deben tener índices.
2. **Evitar funciones complejas** en las expresiones de política.
3. **Ejecutar `ANALYZE`** regularmente en tablas con RLS.
4. **Combinar con particionamiento** para mejorar rendimiento en tablas grandes.

---

### 3.4. Protección contra `search_path` (CVE-2018-1058 y mitigaciones)

La vulnerabilidad CVE-2018-1058 demostró cómo un usuario malicioso podía crear funciones troyanas en el esquema `public` que secuestraran consultas de otros usuarios .

#### El Problema

Por defecto, `search_path = "$user", public`. Un atacante podía crear:

sql

```
-- Función maliciosa en public (no requiere privilegios)
CREATE FUNCTION lower(varchar) RETURNS text AS $$
    SELECT 'Datos robados: ' || $1;
$$ LANGUAGE SQL;
```

Cuando otro usuario ejecutaba `SELECT lower(columna) FROM tabla`, llamaba a la función maliciosa.

#### Soluciones en PostgreSQL 18

1. **Eliminar privilegios CREATE del esquema `public`** (recomendado):

   sql

   ```
   REVOKE CREATE ON SCHEMA public FROM PUBLIC;
   ```

2. **Usar esquemas dedicados por aplicación**:

   sql

   ```
   CREATE SCHEMA app_schema;
   GRANT USAGE ON SCHEMA app_schema TO app_role;
   ALTER ROLE app_role SET search_path TO app_schema;
   ```

3. **Fijar `search_path` por rol**:

   sql

   ```
   ALTER ROLE usuario_confiable SET search_path = '$user', 'app_schema', 'public';
   ```

---

### 3.5. Resumen de Buenas Prácticas de Seguridad en PostgreSQL 18

| Área | Práctica Recomendada |
|---|---|
| **Autenticación** | Usar `scram-sha-256` o `oauth`; nunca `trust` o `password` en producción |
| **Roles** | Usar `CREATEROLE` en lugar de `SUPERUSER` para delegar administración |
| **RLS** | Activar en tablas con datos multi-tenant o datos sensibles |
| **search_path** | Revocar `CREATE` en `public`; definir esquemas explícitos |
| **Monitoreo** | Crear rol con `pg_monitor` para herramientas de observabilidad |
| **Mantenimiento** | Usar `pg_maintain` para operaciones de VACUUM/REINDEX |
| **Backups** | Usar `pg_read_all_data` para roles de respaldo |

---

## Ejercicio Práctico del Módulo 3

sql

```
-- 1. Configurar autenticación OAuth (ejemplo en pg_hba.conf)
```

```
-- host    appdb    app_user    0.0.0.0/0    oauth
issuer=https://auth.mycompany.com scope="openid"

-- 2. Crear roles con privilegios específicos
CREATE ROLE auditor WITH LOGIN PASSWORD 'audit2024';
GRANT pg_read_all_data TO auditor;
GRANT pg_monitor TO auditor;

-- 3. Configurar RLS en tabla de clientes
CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    region TEXT NOT NULL,
    nombre TEXT,
    telefono TEXT
);

ALTER TABLE clientes ENABLE ROW LEVEL SECURITY;

CREATE POLICY clientes_region_policy ON clientes
    USING (region = current_setting('app.user_region'));

-- 4. Verificar políticas activas
SELECT schemaname, tablename, policyname, permissive, roles, cmd, qual
FROM pg_policies
WHERE tablename = 'clientes';
```

---

## Resumen del Módulo 3

Has aprendido que en PostgreSQL 18:

- **OAuth 2.0** es la gran novedad en autenticación, permitiendo integración con proveedores de identidad externos .
- Los **roles predefinidos** (`pg_maintain`, `pg_monitor`, etc.) ofrecen delegación segura de tareas administrativas .
- **RLS** con políticas `PERMISSIVE` y `RESTRICTIVE` permite control de acceso granular a nivel de fila .
- El privilegio `MAINTAIN` está completamente consolidado y seguro tras las correcciones de `search_path` .
- La protección contra **CVE-2018-1058** requiere gestionar cuidadosamente el esquema `public` y el `search_path` .

ahora el modulo 4

Leer 10 páginas web

A continuación se desarrolla el **Módulo 4: Mantenimiento y Optimización (`VACUUM` y `ANALYZE`)** completamente actualizado para **PostgreSQL 18**.

Este módulo es fundamental para todo DBA, ya que un mantenimiento deficiente es la causa principal de degradación del rendimiento en PostgreSQL. Con las mejoras introducidas en

PostgreSQL 18, ahora tenemos herramientas sin precedentes para diagnosticar y optimizar estas operaciones.

---

# Módulo 4: Mantenimiento y Optimización (VACUUM y ANALYZE) en PostgreSQL 18

El motor de PostgreSQL utiliza MVCC (Multiversion Concurrency Control) para gestionar la concurrencia. Esto significa que cuando se actualiza o elimina una fila, la versión anterior (tupla muerta o "dead tuple") permanece en la tabla hasta que se limpia. Sin esta limpieza, la base de datos se hincha ("bloat"), las consultas se ralentizan y los índices se vuelven ineficientes.

---

### 4.1. El Problema Fundamental: Tuplas Muertas y MVCC

**Concepto:** Cada `UPDATE` crea una nueva versión de la fila. La versión antigua es una "tupla muerta". Las tuplas muertas son visibles para transacciones más antiguas que necesitan consistencia, pero una vez que ninguna transacción activa las necesita, deben ser eliminadas.

**Consecuencias del descuido:**

1. **Bloat (Hinchazón):** La tabla ocupa más espacio en disco del necesario.
2. **Rendimiento de Escaneo:** Las consultas que escanean tablas leen muchas páginas vacías.
3. **Transacciones (XID Wraparound):** PostgreSQL tiene un límite de ~2 mil millones de transacciones. Si no se "congelan" las tuplas antiguas, la base de datos se detendrá para protegerse.

---

### 4.2. `VACUUM`: El Recolector de Basura

`VACUUM` es la operación que elimina tuplas muertas y marca el espacio como reutilizable.

#### `VACUUM` Normal (o Concurrente)

- **Función:** Escanea páginas, elimina tuplas muertas, actualiza el Mapa de Visibilidad (Visibility Map), y trunca páginas vacías al final del archivo.
- **Ventaja:** No bloquea lecturas o escrituras (solo toma un bloqueo muy leve).
- **Desventaja:** No compacta la tabla; el espacio liberado queda disponible para futuros `INSERT` pero no se devuelve al sistema operativo.
- **Ejemplo:** `VACUUM (VERBOSE, ANALYZE) nombre_tabla;`

#### `VACUUM FULL` (Reconstrucción)

- **Función:** Reconstruye la tabla completamente desde cero, eliminando todo el espacio desperdiciado y devolviéndolo al sistema operativo.
- **Ventaja:** Reduce drásticamente el tamaño de la tabla.

- **Desventaja: Bloquea la tabla con un `ACCESS EXCLUSIVE LOCK`** durante la operación, lo que detiene todas las operaciones de lectura y escritura.
- **Advertencia:** No debe ejecutarse de forma rutinaria en producción, solo en ventanas de mantenimiento programadas.

#### Herramientas Alternativas (sin bloqueo)

Dado que `VACUUM FULL` bloquea la tabla, existen extensiones populares que logran la compactación sin downtime:

| Herramienta | Mecanismo | Requisito | Cuándo Usarla |
|---|---|---|---|
| **`pg_repack`** | Crea una nueva tabla, copia datos, aplica cambios mediante triggers, luego intercambia las tablas. | Clave primaria o única. | Tablas grandes en producción activa. |
| **`pg_squeeze`** | Similar a `pg_repack`, pero usa replicación lógica en lugar de triggers. | Clave primaria o única. | Entornos donde se prefiere minimizar la sobrecarga de triggers. |

---

### 4.3. `ANALYZE`: El Alimentador del Optimizador

`ANALYZE` recolecta estadísticas sobre la distribución de los datos en las columnas. El planificador de consultas (Query Planner) usa estas estadísticas para decidir si usar un índice o hacer un escaneo secuencial.

- **Sin `ANALYZE`:** El planificador toma decisiones ciegas, lo que resulta en consultas extremadamente lentas.
- **Con `ANALYZE`:** El planificador conoce la cardinalidad (cuántas filas devolverá una condición) y elige el plan óptimo.

**Ejecución manual:**

sql

```sql
-- Analizar una tabla específica
ANALYZE pedidos;

-- Analizar toda la base de datos (puede ser pesado)
ANALYZE;
```

---

### 4.4. `AUTOVACUUM`: El Daemon Automático

PostgreSQL incluye un proceso en segundo plano llamado `autovacuum` que ejecuta `VACUUM` y `ANALYZE` automáticamente según la actividad de la tabla.

#### Configuración Básica (postgresql.conf)

ini

```ini
# Activar autovacuum (por defecto está ON)
autovacuum = on
```

```
# Memoria para procesos autovacuum (importante para rendimiento)
autovacuum_work_mem = -1  # -1 significa usar maintenance_work_mem (2GB
recomendado)[citation:5]
```

#### Parámetros de Activación (¿Cuándo se dispara?)

Un worker de `autovacuum` se activa en una tabla cuando se superan ciertos umbrales de tuplas muertas.

**Fórmula Clásica (sigue vigente):**

text

```
Umbral = autovacuum_vacuum_threshold + (autovacuum_vacuum_scale_factor *
número_de_filas)
```

- `autovacuum_vacuum_threshold`: Por defecto 50.
- `autovacuum_vacuum_scale_factor`: Por defecto 0.2 (20%).

#### Novedad en PostgreSQL 18: `autovacuum_vacuum_max_threshold`

En versiones anteriores, las tablas **muy grandes** (ej. 1 TB) tenían un problema: el 20% de 1 TB son 200 GB. `autovacuum` esperaba a que se acumularan 200 GB de tuplas muertas antes de actuar, lo que causaba picos enormes de I/O y bloat extremo.

**La solución en PostgreSQL 18:** El nuevo parámetro `autovacuum_vacuum_max_threshold` establece un límite **máximo absoluto** de tuplas muertas para activar `VACUUM`, independientemente del tamaño de la tabla.

ini

```
# En postgresql.conf (valor por defecto: 100,000,000)
autovacuum_vacuum_max_threshold = 100000000
```

**Ejemplo práctico:**

| Escenario | Antes de PG18 | Con PG18 |
|---|---|---|
| Tabla con 1,000,000,000 filas | Se activa tras 200M tuplas muertas (20%) | Se activa tras 100M tuplas muertas (el `max_threshold` se alcanza antes que el 20%) |
| Tabla con 100,000 filas | Se activa tras 20,020 tuplas muertas (50 + 20%) | Sigue igual, el `max_threshold` (100M) no se alcanza nunca |

**Configuración por tabla (recomendado):**

sql

```
-- Para una tabla de logs enorme, queremos que vacúe más agresivamente
ALTER TABLE logs_pesados SET (
    autovacuum_vacuum_scale_factor = 0.01,      -- 1% en lugar de 20%
    autovacuum_vacuum_max_threshold = 5000000   -- o tras 5 millones de tuplas
);
```

### 4.5. El Nuevo Panel de Control: Monitoreo de Tiempos en PG18

Antes de PostgreSQL 18, los administradores solo podían ver **cuántas veces** se había ejecutado `VACUUM` en una tabla, pero no **cuánto tiempo** había durado. Esto hacía imposible diagnosticar si los problemas de rendimiento eran por `autovacuum`.

#### Nuevas Columnas en `pg_stat_user_tables`

PostgreSQL 18 introduce cuatro nuevas columnas que registran el tiempo total (en milisegundos) dedicado a operaciones de mantenimiento por cada tabla:

| Columna | Descripción |
|---|---|
| `total_vacuum_time` | Tiempo total de `VACUUM` manual |
| `total_autovacuum_time` | Tiempo total de `VACUUM` automático (autovacuum) |
| `total_analyze_time` | Tiempo total de `ANALYZE` manual |
| `total_autoanalyze_time` | Tiempo total de `ANALYZE` automático |

**Consulta 1: Identificar las tablas que más tiempo consumen en mantenimiento**

sql

```
SELECT
    schemaname || '.' || relname AS tabla,
    round(total_vacuum_time / 1000, 2) AS vacuum_manual_seg,
    round(total_autovacuum_time / 1000, 2) AS vacuum_auto_seg,
    round(total_analyze_time / 1000, 2) AS analyze_manual_seg,
    round(total_autoanalyze_time / 1000, 2) AS analyze_auto_seg,
    round((total_autovacuum_time + total_autoanalyze_time)::numeric /
          nullif(total_vacuum_time + total_analyze_time + total_autovacuum_time
+ total_autoanalyze_time, 0) * 100, 2)
          AS porcentaje_auto
FROM pg_stat_user_tables
ORDER BY (total_autovacuum_time + total_vacuum_time) DESC
LIMIT 10;
```

**Interpretación:**

- Si `porcentaje_auto` es alto (>90%) y hay problemas de I/O, `autovacuum` está sobrecargando el sistema.
- Si una tabla tiene valores muy altos en `vacuum_manual_seg`, alguien está ejecutando `VACUUM` manual en horario productivo.

**Consulta 2: Evaluar la eficiencia de `autovacuum` por tabla**

sql

```
SELECT
    relname,
    n_live_tup AS tuplas_vivas,
    n_dead_tup AS tuplas_muertas,
    autovacuum_count AS veces_vacunado,
```

```
    round(total_autovacuum_time / 1000.0 / nullif(autovacuum_count, 0), 2) AS
segundos_por_vacuum,
    autoanalyze_count AS veces_analizado,
    round(total_autoanalyze_time / 1000.0 / nullif(autoanalyze_count, 0), 2) AS
segundos_por_analyze
FROM pg_stat_user_tables
WHERE total_autovacuum_time > 0
ORDER BY segundos_por_vacuum DESC;
```

**Cuándo alarmarse:**

- `segundos_por_vacuum` > 60 segundos: La tabla es muy grande o el I/O es lento. Considera particionar o ajustar `autovacuum_work_mem`.
- `n_dead_tup` > 1,000,000 pero `autovacuum_count` = 0: `autovacuum` no se está activando. Revisa `autovacuum_vacuum_max_threshold`.

---

### 4.6. Mejoras Adicionales en PostgreSQL 18

#### Eager Freeze Mejorado

PostgreSQL 18 permite que `VACUUM` congele páginas incluso si son "todas visibles" (all-visible), lo que reduce la necesidad de futuros escaneos completos de la tabla para prevenir XID wraparound.

sql

```
-- Controlar qué tan agresivo es este comportamiento (a nivel servidor o tabla)
SET vacuum_max_eager_freeze_failure_rate = 0.1;  -- 0.1% es el valor por defecto
```

#### `vacuum_truncate` como Parámetro

Ahora puedes deshabilitar la truncación de páginas vacías al final de la tabla sin afectar otras operaciones:

sql

```
-- Evitar que VACUUM trunque el archivo (útil si hay locks conflictivos)
ALTER TABLE mi_tabla SET (vacuum_truncate = false);
```

#### Nuevo Rol `pg_signal_autovacuum_worker`

Si un worker de `autovacuum` está consumiendo muchos recursos y afectando el rendimiento, los miembros de este nuevo rol pueden detenerlo.

sql

```
-- Otorgar permiso para gestionar workers de autovacuum
GRANT pg_signal_autovacuum_worker TO dba_operaciones;

-- (Desde el servidor, no vía SQL directo)
-- pg_terminate_backend() se puede usar contra el PID del worker
```

---

### 4.7. Guía de Optimización de Parámetros (PostgreSQL 18)

Basado en las mejores prácticas y las novedades de PG18:

| Parámetro | Valor Sugerido | Impacto |
|---|---|---|
| `maintenance_work_mem` | 2GB - 10% de RAM | Acelera `VACUUM` y `CREATE INDEX` |
| `autovacuum_work_mem` | -1 (usa `maintenance_work_mem`) | Permite que `autovacuum` use más memoria y termine más rápido |
| `autovacuum_vacuum_cost_delay` | 2ms (por defecto) | Controla cuánto "duerme" el worker para no saturar I/O |
| `autovacuum_vacuum_cost_limit` | 200 - 2000 | Mayor valor = worker más agresivo |
| `autovacuum_vacuum_max_threshold` | 100,000,000 (default) | **Nuevo en PG18**. Evita bloat en tablas enormes |
| `vacuum_buffer_usage_limit` | 256MB - 2GB | Limita el uso del buffer pool por `VACUUM` (desde PG16) |

**Ejemplo de configuración robusta para OLTP:**

ini

```
# postgresql.conf
maintenance_work_mem = 2GB
autovacuum_work_mem = -1
autovacuum_vacuum_scale_factor = 0.05        # Más agresivo en tablas medianas
autovacuum_vacuum_max_threshold = 50000000   # Límite absoluto
autovacuum_vacuum_cost_delay = 2ms
autovacuum_vacuum_cost_limit = 1000
vacuum_buffer_usage_limit = 512MB
```

### 4.8. Checklist de Monitoreo Diario

sql

```
-- 1. ¿Hay tablas con muchas tuplas muertas?
SELECT schemaname, relname, n_dead_tup, n_live_tup,
       round(100 * n_dead_tup / nullif(n_live_tup, 0), 2) AS dead_ratio,
       last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC;

-- 2. ¿Hay tablas que nunca han sido analizadas?
SELECT schemaname, relname, last_autoanalyze, last_analyze
FROM pg_stat_user_tables
WHERE last_autoanalyze IS NULL AND last_analyze IS NULL
LIMIT 10;

-- 3. ¿Hay tablas en riesgo de XID wraparound?
SELECT datname, age(datfrozenxid) AS edad_transaccional
FROM pg_database
ORDER BY edad_transaccional DESC;
-- Si edad > 500,000,000, ejecutar VACUUM FREEZE urgente
```

### Resumen del Módulo 4

Has aprendido que en PostgreSQL 18:

- **`VACUUM`** elimina tuplas muertas (no bloquea) y **`VACUUM FULL`** compacta (bloquea).
- **`AUTOVACUUM`** es automático pero requiere ajustes finos.
- **`autovacuum_vacuum_max_threshold`** es la gran novedad para prevenir bloat en tablas enormes.
- Las nuevas columnas de tiempo (`total_autovacuum_time`) permiten diagnosticar el impacto en rendimiento.
- Herramientas como `pg_repack` permiten compactar sin downtime cuando `VACUUM FULL` no es viable.
- Un DBA debe monitorear diariamente tuplas muertas, estadísticas y edad transaccional.

desarrolame los 3 modulos que quedan

Leer 8 páginas web

A continuación se desarrollan los **Módulos 5, 6 y 7** (Copias de Seguridad, Replicación y Monitoreo/Auditoría) completos, actualizados con las novedades de **PostgreSQL 18**.

---

# Módulo 5: Copias de Seguridad (Backup) y Recuperación (PITR)

La estrategia de respaldo es el pilar de cualquier administración de bases de datos. PostgreSQL 18 introduce mejoras significativas, destacando especialmente el **backup incremental nativo**.

## 5.1. Backup Lógico (`pg_dump` / `pg_dumpall`)

El respaldo lógico extrae las definiciones de esquemas y datos en un formato de comandos SQL o archivos planos.

| Herramienta | Alcance | Uso Típico |
|---|---|---|
| **`pg_dump`** | Una base de datos específica | Migraciones, backups selectivos |
| **`pg_dumpall`** | Todo el cluster (roles, tablespaces, BBDD) | Respaldos completos de instancias pequeñas |

**Ejemplo de uso**:

bash

```
# Backup en formato custom (comprimido)
pg_dump -Fc -U postgres -d mi_bd > mi_bd.dump

# Restaurar desde backup custom
pg_restore -U postgres -d mi_bd_nueva mi_bd.dump
```

**Limitación**: Los backups lógicos son más lentos que los físicos para bases de datos grandes y no permiten recuperación punto-a-tiempo (PITR).

## 5.2. Backup Físico Tradicional (`pg_basebackup`)

`pg_basebackup` crea una copia binaria exacta del cluster, ideal para configurar réplicas en caliente (hot standby).

bash

```
# Backup físico completo desde un servidor en ejecución
pg_basebackup -h servidor_principal -D /ruta/backup -U replicador -v -P --wal-method=stream
```

## 5.3. Novedad Estrella: Backup Incremental Nativo (PostgreSQL 18)

PostgreSQL 18 introduce **backups incrementales a nivel de archivo** de forma nativa, eliminando la dependencia de herramientas externas como `pgBackRest` para esta funcionalidad básica.

### Concepto

Un backup incremental solo almacena los bloques de datos que cambiaron desde el backup anterior, reduciendo drásticamente el tiempo de respaldo y el espacio de almacenamiento.

### Herramientas Asociadas

| Herramienta | Función |
|---|---|
| **`pg_basebackup`** (mejorado) | Ahora soporta la opción `--incremental` para crear backups incrementales |
| **`pg_combinebackup`** (nueva) | Reconstruye un backup completo sintético a partir de una cadena de backups (full + incrementales) |

### Flujo de Trabajo Paso a Paso

**Paso 1: Realizar un Backup Full (Base)**

bash

```
# Backup completo inicial
pg_basebackup -D /backups/base_full -Ft -z -P
```

**Paso 2: Realizar Backups Incrementales**

bash

```
# Backup incremental basado en el backup anterior
pg_basebackup -D /backups/inc1 --incremental=/backups/base_full -Ft -z -P
# Posteriormente, otro incremental
pg_basebackup -D /backups/inc2 --incremental=/backups/inc1 -Ft -z -P
```

**Paso 3: Reconstruir el Backup Completo para Restauración**

bash

```
# Combinar full + incrementales en orden (de más antiguo a más nuevo)
pg_combinebackup /backups/base_full /backups/inc1 /backups/inc2 -o /backups/restore_full
```

El parámetro `-o` especifica el directorio de salida donde se generará el backup sintético completo.

**Opciones Clave de `pg_combinebackup`**

| Opción | Efecto | Cuándo Usarla |
|---|---|---|
| `-k` / `--link` | Usa enlaces duros en lugar de copiar archivos | Para ahorrar espacio en disco, pero los directorios de entrada no deben modificarse después |
| `--clone` | Usa clonación de archivos (reflinks) en sistemas compatibles (Btrfs, XFS, APFS) | Para reconstrucción casi instantánea en sistemas de archivos modernos |
| `-n` / `--dry-run` | Simula la operación sin escribir archivos | Para verificar la cadena de backups antes de ejecutar |

**Limitación Importante**

`pg_combinebackup` **no recalcula las checksums de página** al escribir el directorio de salida. Si el backup final se tomó con checksums habilitadas pero alguno de los incrementales no, el directorio resultante puede contener páginas con checksums inválidas. La solución es crear un nuevo backup full después de cambiar el estado de checksums del cluster.

## 5.4. Recuperación Punto-a-Tiempo (PITR) con WAL

La recuperación punto-a-tiempo utiliza los archivos WAL (Write-Ahead Logs) para restaurar un cluster a un momento específico.

**Componentes Necesarios**

1. Un **backup base** (físico completo)
2. Todos los **archivos WAL** generados desde el backup hasta el momento objetivo

**Configuración para PITR**

En `postgresql.conf` (o `recovery.conf` en versiones modernas integrado):

ini

```
# Modo de recuperación
restore_command = 'cp /wal_archive/%f %p'

# Punto de recuperación (usar uno de estos)
recovery_target_time = '2024-12-25 14:30:00'  # Tiempo específico
recovery_target_xid = '1234567'                # ID de transacción
recovery_target_lsn = '0/3000000'             # LSN específico

# Acción al alcanzar el target
recovery_target_action = 'promote'  # promote = convertirse en primary
```

**Proceso de Recuperación**

1. Detener el servidor PostgreSQL.
2. Restaurar el backup base en el directorio de datos.
3. Configurar los parámetros de recuperación.

4. Iniciar el servidor en modo recovery.
5. Verificar la recuperación y eliminar los archivos de configuración de recovery.

---

# Módulo 6: Réplica Lógica y Física

La replicación es esencial para alta disponibilidad, escalabilidad de lectura y migraciones. PostgreSQL 18 introduce `pg_createsubscriber` para simplificar drásticamente la conversión de réplicas.

## 6.1. Streaming Replication (Réplica Física)

La replicación física copia el cluster completo bloque por bloque desde el primario al standby.

### Configuración Rápida

**En el primario (`postgresql.conf`)**:

ini

```
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
```

**En el primario (`pg_hba.conf`)**:

bash

```
host replication replicador 192.168.1.0/24 scram-sha-256
```

**Crear usuario de replicación**:

sql

```
CREATE USER replicador WITH REPLICATION LOGIN PASSWORD 'secure_pass';
```

**Crear el standby**:

bash

```
pg_basebackup -h primary_ip -D /var/lib/postgresql/18/standby -U replicador -P
--wal-method=stream
```

**Configurar el standby (`postgresql.conf`)**:

ini

```
primary_conninfo = 'host=primary_ip port=5432 user=replicador
password=secure_pass'
primary_slot_name = 'standby_slot'
hot_standby = on
```

## 6.2. Logical Replication (Réplica Lógica)

La replicación lógica opera a nivel de fila, permitiendo replicar tablas específicas entre diferentes versiones de PostgreSQL.

**Componentes Clave**

| Concepto | Rol | Ubicación |
|---|---|---|
| **Publicación (Publication)** | Define qué tablas se replican | Servidor Publicador (Publisher) |
| **Suscripción (Subscription)** | Recibe los datos de una publicación | Servidor Suscriptor (Subscriber) |
| **Replication Slot** | Mantiene el punto de replicación | Servidor Publicador |

**Creación de una Réplica Lógica**

**En el publicador**:

sql

```
-- Crear publicación para todas las tablas
CREATE PUBLICATION mi_publicacion FOR ALL TABLES;

-- O para tablas específicas
CREATE PUBLICATION mi_publicacion FOR TABLE usuarios, pedidos;
```

**En el suscriptor**:

sql

```
-- Crear suscripción
CREATE SUBSCRIPTION mi_suscripcion
    CONNECTION 'host=publicador_ip dbname=mi_bd user=replicador password=pass'
    PUBLICATION mi_publicacion;
```

**Parámetros Importantes de `CREATE SUBSCRIPTION`**

| Parámetro | Valor por Defecto | Descripción |
|---|---|---|
| `copy_data` | `true` | Copia los datos existentes al iniciar la replicación |
| `enabled` | `true` | Activa la replicación inmediatamente |
| `create_slot` | `true` | Crea automáticamente el replication slot |
| `two_phase` | `false` | Habilita confirmación en dos fases para transacciones preparadas |
| `streaming` | `parallel` | Controla cómo se aplican transacciones largas |

## 6.3. Novedad: `pg_createsubscriber` (PostgreSQL 18)

Esta nueva utilidad **convierte una réplica física (standby) en una réplica lógica** sin necesidad de volcar todos los datos.

**El Problema que Resuelve**

Tradicionalmente, configurar una réplica lógica requería:

1. Hacer un volcado inicial de los datos (`copy_data = true`).
2. Transferir grandes volúmenes de datos.
3. Aplicar los cambios acumulados durante la transferencia.

Para bases de datos grandes (TB), este proceso podía tomar días.

### La Solución en PG18

`pg_createsubscriber` aprovecha que el standby físico ya tiene todos los datos sincronizados. Simplemente "convierte" ese standby en un nodo de replicación lógica.

### Flujo de Trabajo

bash

```
# En el servidor que será el suscriptor (standby físico)
pg_createsubscriber \
    -D /var/lib/postgresql/18/standby \
    -P "host=publicador_ip dbname=mi_bd user=admin password=pass" \
    -d mi_bd \
    --enable-two-phase
```

### Prerrequisitos

| Requisito | Descripción |
|---|---|
| Misma versión mayor | Publicador y suscriptor deben ejecutar PostgreSQL 18 |
| `wal_level = logical` | Configurado en el publicador |
| Espacio para replication slots | `max_replication_slots` suficiente en el publicador |
| Conexión local | El suscriptor debe aceptar conexiones locales |
| Standby físico | El directorio destino debe ser una réplica física |

### Opciones Útiles

| Opción | Función |
|---|---|
| `-a` / `--all` | Convierte todas las bases de datos (excluyendo templates) |
| `--enable-two-phase` | Habilita confirmación en dos fases (útil para migraciones seguras) |
| `-n` / `--dry-run` | Simula la conversión sin aplicar cambios |
| `--clean=publications` | Limpia publicaciones existentes antes de crear nuevas |

### Consideraciones de Seguridad

- **DDL no se replica**: Evita ejecutar comandos DDL (ALTER TABLE, CREATE SCHEMA) durante la conversión.
- **Fallos post-promoción**: Si `pg_createsubscriber` falla después de promover el standby, se recomienda crear un nuevo standby desde cero.
- **WAL retention**: Configurar `max_slot_wal_keep_size = -1` en el publicador para evitar la eliminación prematura de WAL.

### 6.4. Comparativa: Réplica Física vs. Lógica

| Característica | Réplica Física | Réplica Lógica |
|---|---|---|
| **Nivel de replicación** | Cluster completo | Tablas específicas |
| **Versiones de PostgreSQL** | Misma versión mayor | Diferentes versiones posibles |
| **Replicación bidireccional** | No | Sí (con configuraciones avanzadas) |
| **DDL en suscriptor** | No permitido | Permitido (puede causar conflictos) |
| **Caso de uso típico** | HA, Disaster Recovery | Migraciones, Data Warehousing |

---

# Módulo 7: Monitoreo y Auditoría

El monitoreo efectivo y la auditoría son esenciales para mantener el rendimiento y la conformidad regulatoria.

## 7.1. Estadísticas en Tiempo Real

PostgreSQL expone estadísticas del sistema a través de vistas del esquema `pg_stat_*`.

#### Vistas Fundamentales para el DBA

| Vista | Información que Proporciona |
|---|---|
| **`pg_stat_activity`** | Conexiones activas, consultas en ejecución, estado de transacciones |
| **`pg_stat_user_tables`** | Escaneos, lecturas/escrituras, tuplas vivas/muertas por tabla |
| **`pg_stat_bgwriter`** | Actividad del proceso background writer (checkpoints, buffers escritos) |
| **`pg_stat_replication`** | Estado de la replicación física (standbys conectados, lag) |
| **`pg_stat_subscription`** | Estado de suscripciones de replicación lógica |

#### Consultas de Diagnóstico Comunes

**1. Identificar consultas bloqueantes**:

sql

```
SELECT
    blocked.pid AS blocked_pid,
    blocking.pid AS blocking_pid,
    blocked.query AS blocked_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
    ON blocked.wait_event_type = 'Lock'
    AND blocking.pid = ANY(pg_blocking_pids(blocked.pid));
```

**2. Ver tablas con más tuplas muertas**:

sql

```sql
SELECT relname, n_dead_tup, n_live_tup,
       round(100 * n_dead_tup / nullif(n_live_tup, 0), 2) AS dead_ratio
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC;
```

## 7.2. Logging Avanzado

La configuración adecuada del log es crucial para la resolución de problemas.

### Configuración Recomendada (`postgresql.conf`)

ini

```ini
# Qué loguear
log_destination = 'stderr'
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'

# Contenido del log
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
log_statement = 'ddl'  # Solo loguear DDL (CREATE, ALTER, DROP)
log_min_duration_statement = 1000  # Log de consultas > 1 segundo

# Errores y locks
log_lock_waits = on
log_temp_files = 0  # Log de archivos temporales
```

## 7.3. Auditoría con `pgAudit`

Para cumplir con normativas como GDPR, SOX o PCI-DSS, se recomienda la extensión **pgAudit** (disponible como contrib).

### Instalación y Configuración

sql

```sql
-- Crear la extensión
CREATE EXTENSION pgaudit;

-- Configurar sesión
SET pgaudit.log = 'read, write, ddl, role';
SET pgaudit.log_catalog = on;
SET pgaudit.log_relation = on;
SET pgaudit.log_parameter = on;  -- Incluye parámetros (¡cuidado con datos
sensibles!)
```

### Niveles de Auditoría

| Nivel | Qué Audita |
|---|---|
| `READ` | SELECT, COPY |
| `WRITE` | INSERT, UPDATE, DELETE, TRUNCATE |
| `DDL` | CREATE, ALTER, DROP |
| `ROLE` | GRANT, REVOKE, CREATE/ALTER/DROP ROLE |

| Nivel | Qué Audita |
|---|---|
| `MISC` | Otros comandos administrativos |

## 7.4. Nuevas Vistas de Monitoreo en PostgreSQL 18

| Vista | Descripción |
|---|---|
| `pg_stat_checkpointer` | Métricas detalladas del proceso checkpointer (tiempos, frecuencia) |
| `pg_wait_events` | Documentación y consulta de eventos de espera del sistema |

## 7.5. Checklist de Monitoreo Diario

sql

```
-- 1. Conexiones activas por base de datos
SELECT datname, count(*) AS conexiones_activas
FROM pg_stat_activity
WHERE state = 'active'
GROUP BY datname;

-- 2. Transacciones largas (más de 5 minutos)
SELECT pid, usename, query,
       age(now(), xact_start) AS duracion_transaccion
FROM pg_stat_activity
WHERE state = 'idle in transaction'
  AND xact_start < now() - interval '5 minutes';

-- 3. Tamaño de las tablas (con bloat estimado)
SELECT schemaname, tablename,
       pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS
tamaño_total
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC
LIMIT 10;

-- 4. Último vacuum/analyze por tabla
SELECT schemaname, relname,
       last_vacuum, last_autovacuum,
       last_analyze, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY COALESCE(last_autovacuum, last_vacuum) DESC NULLS LAST
LIMIT 10;
```

---

## Resumen de los Módulos 5, 6 y 7

| Módulo | Tema Principal | Novedades Clave en PostgreSQL 18 |
|---|---|---|
| **5** | Copias de Seguridad | Backup incremental nativo con `pg_basebackup --incremental` y `pg_combinebackup` |
| **6** | Replicación | `pg_createsubscriber` para convertir réplicas físicas en lógicas sin volcar datos |

| Módulo | Tema Principal | Novedades Clave en PostgreSQL 18 |
|---|---|---|
| **7** | Monitoreo | Nuevas vistas `pg_stat_checkpointer` y `pg_wait_events`; mejora en logs y `pgAudit` |

Estos tres módulos completan el curso, proporcionando una base sólida para administrar entornos PostgreSQL 18 en producción, desde la estrategia de respaldo hasta el monitoreo continuo y la auditoría de seguridad.

y por ultimo en modulo 8

Leer 10 páginas web

A continuación se desarrolla el **Módulo 8: Parámetros Avanzados y Ajustes de PostgreSQL 18**, que cierra el curso. Este módulo se centra en las nuevas capacidades de tuning fino que ofrece esta versión, cubriendo desde la gestión de memoria interna hasta el control del `AUTOVACUUM` y la ejecución de comandos de mantenimiento.

---

## Módulo 8: Parámetros Avanzados y Ajustes de Postgres 18

Como DBA, pasar de una configuración funcional a una optimizada es lo que marca la diferencia. PostgreSQL 18 introduce una serie de parámetros y ajustes que te otorgan un control sin precedentes sobre el rendimiento y el comportamiento del motor.

---

### 8.1. Nuevos Parámetros de Configuración en 18

#### 8.1.1. Control de Memoria Fina: Buffers SLRU Configurables

**¿Qué son los SLRUs?** Los "Simple Least-Recently-Used" buffers son estructuras de memoria interna que PostgreSQL utiliza para gestionar metadatos críticos del sistema (como el estado de las transacciones o las notificaciones). Antes de PostgreSQL 18, su tamaño estaba fijo. Ahora son totalmente configurables.

Estos parámetros te permiten dedicar más memoria a ciertas estructuras internas en sistemas con mucha carga o reducir su huella en sistemas con recursos limitados.

**Ubicación en `postgresql.conf`:**
Ahora se encuentran bajo `Resource Usage / Memory` (en lugar de "File Locations" como antes).

| Parámetro | Descripción | Valor por Defecto | Cuándo Ajustarlo |
|---|---|---|---|
| **`commit_timestamp_buffers`** | Almacena tiempos de confirmación (`pg_commit_ts`). | 0 (automático) | Sistemas que usan `track_commit_timestamp` o réplica lógica pesada. |
| **`multixact_off`** | Almacena offsets para | 16 | Si hay muchos bloqueos de fila |

| Parámetro | Descripción | Valor por Defecto | Cuándo Ajustarlo |
|---|---|---|---|
| **`set_buffers`** | transacciones Multixact. | | compartidos (`FOR SHARE`). |
| **`multixact_member_buffers`** | Almacena miembros de transacciones Multixact. | 32 | Similar a `offset`, para workloads con muchas transacciones concurrentes. |
| **`notify_buffers`** | Gestiona la cola de `NOTIFY`/`LISTEN`. | 16 | Sistemas con uso intensivo de eventos Pub/Sub. |
| **`serializable_buffers`** | Para transacciones aisladas con `SERIALIZABLE`. | 32 | Si usas el nivel de aislamiento más estricto. |
| **`subtransaction_buffers`** | Para subtransacciones (ej. `SAVEPOINT`). | 0 (automático) | Aplicaciones que usan muchos savepoints. |
| **`transaction_buffers`** | Almacena el estado de las transacciones (`pg_xact`). | 0 (automático) | Sistemas con alta tasa de commits/rollbacks. |

**Nota:** Todos estos cambios requieren reinicio del servidor.

---

#### 8.1.2. Ajuste Dinámico de `AUTOVACUUM` (Sin Reiniciar)

Anteriormente, cambiar el número de workers de `autovacuum` requería un reinicio. En PostgreSQL 18, se separa la reserva de slots del límite concurrente.

- **`autovacuum_worker_slots`**: El número total de workers que el sistema puede crear (requiere reinicio). Piensa en esto como el "pool" de workers.
- **`autovacuum_max_workers`**: El número de workers que pueden ejecutarse **simultáneamente** (se puede recargar con `pg_reload_conf()`). Este valor no puede exceder el número de `slots`.

  **Beneficio:** Puedes aumentar temporalmente la intensidad del `autovacuum` durante horas de baja actividad sin planificar un reinicio.

  ini

```
# postgresql.conf
autovacuum_worker_slots = 10   # Requiere reinicio
autovacuum_max_workers = 5     # Recargable en caliente
```

---

#### 8.1.3. Parámetros de Rendimiento de I/O

- **`io_method` y `io_workers` (Novedad en 18)**
  PostgreSQL 18 introduce un subsistema de I/O Asíncrono (AIO). Esto permite que `VACUUM` (entre otros procesos) encargue la lectura de páginas al sistema operativo y mientras espera, siga procesando datos en lugar de quedarse bloqueado.
- `io_method`: Define el mecanismo (`worker` es el nuevo valor por defecto).

- `io_workers`: Número de workers para I/O asíncrono (por defecto 3). Aumentar este valor puede acelerar `VACUUM` en sistemas con discos rápidos (NVMe/SSD).
- **`vacuum_truncate` (Control Explícito)**
  Cuando `VACUUM` limpia el final de una tabla, necesita un bloqueo `ACCESS EXCLUSIVE`. En réplicas, esto puede ralentizar las lecturas. Ahora puedes desactivar la truncación automática.

  ini

  ```
  # postgresql.conf
  vacuum_truncate = off   # Previene la truncación automática
  ```

  Luego, ejecutas la truncación manualmente en ventanas de mantenimiento:

  sql

  ```
  VACUUM (TRUNCATE ON) nombre_tabla;
  ```

---

#### 8.1.4. Eager Freeze Controlado (`vacuum_max_eager_freeze_failure_rate`)

Para evitar el temido "wraparound" (transacciones agotadas), PostgreSQL necesita congelar tuplas viejas. En versiones anteriores, esto a menudo significaba un `VACUUM` masivo y bloqueante. En PostgreSQL 18, el `VACUUM` normal puede "congelar agresivamente" las páginas, distribuyendo la carga.

Este parámetro controla qué porcentaje de páginas "todas visibles" pueden fallar en congelarse antes de que el `VACUUM` se rinda. Un valor más bajo (ej. 0.01) hace que el `VACUUM` sea más persistente en congelar.

ini

```
vacuum_max_eager_freeze_failure_rate = 0.1  # Valor por defecto en 18
```

---

### 8.2. El Privilegio `MAINTAIN` y el Rol `pg_maintain` (Consolidados)

Aunque se introdujo en versiones anteriores, hubo un *revival* y consolidación importante camino a PostgreSQL 18 debido a problemas de seguridad con `search_path`. Ahora está completamente seguro y es la forma estándar de delegar mantenimiento.

**¿Qué permite?**
El privilegio `MAINTAIN` permite ejecutar comandos como `VACUUM`, `ANALYZE`, `REINDEX`, `CLUSTER` y `REFRESH MATERIALIZED VIEW` sobre una tabla **sin ser el dueño de la misma**.

**Rol `pg_maintain`:**
Es un rol predefinido que, al ser asignado a un usuario, le otorga el privilegio `MAINTAIN` sobre **todas las relaciones** del sistema, así como el permiso de `USAGE` implícito en todos los esquemas para propósitos de mantenimiento.

**Caso práctico:**

sql

```
-- 1. El dueño es 'sistema_ventas'
CREATE TABLE facturas (id int, total int);
INSERT INTO facturas VALUES (1,100);

-- 2. El DBA 'juan' NO es dueño
CREATE USER juan WITH LOGIN PASSWORD 'pass';
-- juan no puede hacer vacuum directamente (no es dueño)

-- 3. Delegamos el mantenimiento
GRANT pg_maintain TO juan;

-- Ahora 'juan' puede ejecutar:
-- VACUUM facturas;
-- ANALYZE facturas;
-- REINDEX TABLE facturas;
```

---

### 8.3. Mejoras en `ANALYZE` y Particionamiento

En PostgreSQL 18, el comportamiento por defecto de `VACUUM` y `ANALYZE` ahora es **recursivo** para tablas heredadas (particionamiento por herencia), igualando el comportamiento del particionamiento declarativo.

- **Por defecto:** `VACUUM pariente;` también procesa los hijos.
- **Exclusividad:** Si solo quieres procesar la tabla padre (ej. para actualizar estadísticas sin tocar los hijos masivos), usa la palabra clave `ONLY`.

sql

```
-- Antes (PG16): Había que hacerlo manual o con scripts.
-- Ahora (PG18):
VACUUM pariente;               -- Vacía padre E hijos
ANALYZE ONLY pariente;        -- Analiza SOLO el padre (útil para tablas
gigantes)
```

---

### 8.4. Checklist de Configuración Inicial para PG18

Basado en las novedades, aquí tienes una configuración recomendada para empezar:

ini

```
# ==========================================
# RECOMENDACIONES POSTGRESQL 18
# ==========================================

# --- 1. MEMORIA (SLRUs) ---
# Si tienes RAM y alta concurrencia, aumentar estos valores ayuda.
commit_timestamp_buffers = 64
notify_buffers = 64

# --- 2. MANTENIMIENTO (VACUUM) ---
# Límite duro para tablas enormes (evita el Bloat)
autovacuum_vacuum_max_threshold = 100000000  # 100M de tuplas muertas

# Control granular de la truncación (evita locks en replicas)
vacuum_truncate = off
```

```
# Congelado agresivo controlado (distribuye la carga)
vacuum_max_eager_freeze_failure_rate = 0.05

# --- 3. I/O ASINCRONO (AIO) ---
# Activa workers de I/O para VACUUM más rápido
io_method = worker
io_workers = 6   # Ajusta según la velocidad de tu disco (3 es default)

# --- 4. AUTOVACUUM FLEXIBLE ---
# Ajusta la concurrencia en caliente
autovacuum_worker_slots = 20   # Requiere reinicio
autovacuum_max_workers = 10    # Recargable

# --- 5. SEGURIDAD Y DELEGACIÓN ---
# Asegura que los DBAs de mantenimiento NO necesiten ser superuser
# (Esto se asigna via SQL: GRANT pg_maintain TO user;)
```

---

## Resumen Final del Curso

A lo largo de estos 8 módulos, hemos cubierto la evolución de PostgreSQL hasta su versión 18, centrándonos en:

1. **Arquitectura:** Dónde vive y cómo respira PostgreSQL.
2. **Usuarios:** Gestión moderna con Roles y el nuevo `pg_maintain`.
3. **Seguridad:** Desde `pg_hba.conf` hasta RLS y OAuth2.
4. **Mantenimiento:** El nuevo paradigma de `VACUUM` con límites duros y monitoreo de tiempos.
5. **Respaldo:** Backup Incremental Nativo (`pg_combinebackup`).
6. **Replicación:** El puente mágico `pg_createsubscriber`.
7. **Monitoreo:** Estadísticas detalladas para diagnósticos precisos.
8. **Ajuste:** Control fino de la memoria interna y el I/O.

Con estas herramientas, estás listo para administrar entornos PostgreSQL 18 de misión crítica de manera eficiente, segura y moderna.